

.NET Development Exam Preparation Guide

Complete Q&A with Answers | Intern Level

C# Fundamentals

.NET Core

ASP.NET MVC

LINQ

Entity Framework

REST APIs

Exam Duration: 1 hour 31 minutes

itransition | .NET Intern Position

1. C# Fundamentals
2. OOP — The Four Pillars
3. Collections & LINQ
4. .NET Core Concepts
5. ASP.NET Core & REST APIs
6. Entity Framework Core & SQL
7. Quick Reference — Must-Know Answers

Exam Duration: 1 hour 31 minutes

itransition | .NET Intern Position

1. C# Fundamentals

.NET Development

Q1. What is the difference between value types and reference types in C#?

Value types store data directly in memory (stack). Examples: int, float, bool, char, struct, enum.

Reference types store a reference (pointer) to data in the heap. Examples: class, string, array, interface, delegate.

When you assign a value type to another variable, a copy is made. With reference types, both variables point to the same object.

```
int a = 5;
int b = a;    // b is a copy – changing b won't affect a

string s1 = "hello";
string s2 = s1; // both point to same string object
```

■ *Exam Tip: This is one of the most common C# interview questions. Remember: struct = value, class = reference.*

Q2. What is the difference between 'ref' and 'out' parameters?

ref: The variable must be initialized before being passed. The method can read AND write to it.

out: The variable does NOT need to be initialized. The method MUST assign a value before returning.

Both allow a method to modify the caller's variable.

```
void AddFive(ref int x) { x += 5; }    // must init before calling
void GetValue(out int x) { x = 10; }   // must assign inside method
```

■ *Exam Tip: Use 'out' when a method needs to return multiple values.*

Q3. What is the difference between 'abstract class' and 'interface'?

Abstract class: Can have implemented methods, fields, constructors. A class can inherit only ONE abstract class.

Interface: Only method signatures (C# 8+ allows default implementations). A class can implement MULTIPLE interfaces.

Use abstract class when classes share code. Use interface for defining contracts/capabilities.

```
abstract class Animal { public abstract void Speak(); }
interface IFlyable { void Fly(); }
class Bird : Animal, IFlyable { ... } // OK – one class, one interface
```

Q4. What does 'virtual' and 'override' mean in C#?

virtual: Marks a base class method as overrideable by derived classes.

override: In a derived class, replaces the base class virtual method with a new implementation.

Without 'virtual', the base method cannot be properly overridden (you'd use 'new' to hide it instead).

```
class Shape { public virtual double Area() => 0; }  
class Circle : Shape { public override double Area() => Math.PI * r * r; }
```

Q5. What is the difference between '==' and '.Equals()' in C#?

For **primitive/value types**, both compare values — they behave the same.

For **reference types**, '==' compares references (memory addresses) by default, while '.Equals()' compares content if overridden.

String is special: '==' compares content for strings even though string is a reference type.

■ *Exam Tip: Always use .Equals() or string.Compare() for strings when culture/case sensitivity matters.*

Q6. What is exception handling in C#? Explain try/catch/finally.

try: Wraps code that might throw an exception.

catch: Handles the exception if one occurs. You can have multiple catch blocks for different exception types.

finally: Always runs, whether or not an exception occurred — used for cleanup (closing files, DB connections).

```
try {  
    int result = 10 / 0;  
} catch (DivideByZeroException ex) {  
    Console.WriteLine(ex.Message);  
} finally {  
    Console.WriteLine("Always runs");  
}
```

Q7. What is a nullable type in C#?

Value types cannot normally be null. A **nullable type** (T?) allows value types to also hold null. Common for database fields that might be empty.

```
int? age = null;    // nullable int
if (age.HasValue) Console.WriteLine(age.Value);
int result = age ?? 0; // null-coalescing: use 0 if null
```

■ *Exam Tip: The '??' operator returns the left value if not null, otherwise the right value.*

C# Fundamentals

.NET Core

ASP.NET MVC

LINQ

Entity Framework

REST APIs

Exam Duration: 1 hour 31 minutes

itransition | .NET Intern Position

2. OOP — The Four Pillars

.NET Development

Q1. What are the four pillars of OOP?

Encapsulation: Hiding internal data using access modifiers (private, protected). Expose only what is necessary.

Inheritance: A class (child) inherits properties and methods from another class (parent), promoting code reuse.

Polymorphism: One interface, many forms. A method behaves differently based on the object calling it.

Abstraction: Hiding complex implementation details and showing only essential features.

■ *Exam Tip: Remember: EIPA — Encapsulation, Inheritance, Polymorphism, Abstraction.*

Exam Duration: 1 hour 31 minutes

Q2. What is a constructor and what types exist in C#?

A constructor is a special method that runs when an object is created. It has the same name as the class.

Default constructor: No parameters (auto-generated if none defined).

Parameterized constructor: Takes arguments to initialize the object.

Static constructor: Runs once for the class (not per instance), used to initialize static members.

Copy constructor: Takes an object of the same type and copies its values.

```
class Person {  
    public string Name;  
    public Person(string name) { Name = name; } // parameterized  
    static Person() { Console.WriteLine("Class loaded"); } // static  
}
```

Q3. What is the difference between 'static' and 'instance' members?

Instance members: Belong to a specific object. Each object has its own copy.

Static members: Belong to the class itself, not to any object. Shared across all instances.

Static methods cannot access instance (non-static) members directly.

```
class Counter {  
    public int Count = 0; // instance – each object has its own  
    public static int Total = 0; // static – shared across all objects  
}
```

Q4. What is method overloading vs method overriding?

Overloading: Same method name, different parameters (number or type). Resolved at compile time.

Overriding: Child class replaces parent's virtual method. Resolved at runtime (polymorphism).

```
// Overloading (same class)
void Print(int x) { }
void Print(string s) { }

// Overriding (parent → child)
class A { public virtual void Show() { } }
class B : A { public override void Show() { } }
```

■ *Exam Tip: Overloading = compile-time. Overriding = runtime.*

Exam Duration: 1 hour 31 minutes

itransition | .NET Intern Position

3. Collections & LINQ

.NET Development

Q1. What are the main collection types in C#?

Array: Fixed size, fast access by index.

List<T>: Dynamic size, ordered, allows duplicates. Most commonly used.

Dictionary<TKey, TVal>: Key-value pairs, fast lookup by key, keys must be unique.

HashSet<T>: Unordered collection of unique items.

Queue<T>: First-In-First-Out (FIFO). **Stack<T>:** Last-In-First-Out (LIFO).

C# Fun

■ *Exam Tip: List is your go-to. Use Dictionary when you need fast lookup by a key.*

Pls

Q2. What is the difference between IEnumerable and IQueryable?

IEnumerable: Works in memory. Good for LINQ to Objects. Executes query on the client side.

IQueryable: Works with out-of-process sources (like databases). Translates LINQ into SQL — filtering happens on the server.

IQueryable extends IEnumerable.

■ *Exam Tip: For Entity Framework DB queries, use IQueryable so filters run in the database, not in app memory.*

Q3. What is LINQ and write a common example?

LINQ (Language Integrated Query) lets you query collections using SQL-like syntax in C#.

Common operators: Where (filter), Select (transform), OrderBy (sort), GroupBy, First, Any, Count.

```
List nums = new List {1,2,3,4,5,6,7,8,9,10};

// Filter even numbers, square them, order descending
var result = nums
    .Where(n => n % 2 == 0)
    .Select(n => n * n)
    .OrderByDescending(n => n)
    .ToList();
// result: [100, 64, 36, 16, 4]
```


Q4. What is deferred execution in LINQ?

LINQ queries are not executed when defined — they execute when you iterate over the results (e.g., with `foreach`, `ToList()`, `Count()`).

This is called **deferred execution**. It allows efficient query composition.

Methods like `ToList()`, `ToArray()`, `Count()`, `First()` trigger immediate execution.

```
var query = list.Where(x => x > 5); // NOT executed yet
var result = query.ToList();        // executed HERE
```

■ *Exam Tip: Call `.ToList()` to force immediate execution and avoid re-querying the data source.*

C# Fundamentals

.NET Core

ASP.NET MVC

LINQ

Entity Framework

REST APIs

Exam Duration: 1 hour 31 minutes

itransition | .NET Intern Position

4. .NET Core Concepts

.NET Development

Q1. What is the difference between .NET Framework, .NET Core, and .NET 5/8/9?

.NET Framework: Windows-only, older. Still used for legacy apps.

.NET Core: Cross-platform (Windows, Linux, macOS), open-source, modern, faster.

.NET 5, 6, 7, 8, 9: Unified platform — the successor to both. Just called '.NET' now. .NET 8 is the current LTS version.

For new projects, always use the latest .NET (not .NET Framework).

■ *Exam Tip: In interviews, say you prefer .NET 8 (LTS) for new projects.*

Q2. What is the CLR (Common Language Runtime)?

CLR is the runtime environment that manages C# program execution.

It handles: **Memory management** (Garbage Collection), **JIT compilation** (converts IL code to machine code), **Exception handling**, **Thread management**, and **Type safety**.

When you compile C#, it becomes IL (Intermediate Language). CLR then compiles IL to native machine code at runtime.

Q3. What is Garbage Collection in .NET?

The Garbage Collector (GC) automatically reclaims memory from objects that are no longer referenced.

Objects are organized in **generations**: Gen 0 (new objects), Gen 1 (survived one collection), Gen 2 (long-lived).

You don't need to free memory manually in C# (unlike C/C++). GC does it for you.

You can implement **IDisposable** and use the 'using' statement for unmanaged resources (files, DB connections).

```
using (var reader = new StreamReader("file.txt"))
{
    string text = reader.ReadToEnd();
} // reader.Dispose() called automatically here
```

Q4. What is async/await in C#?

async/await is C#'s way of writing asynchronous code that looks synchronous.

An **async** method runs asynchronously and returns Task or Task<T>.

await pauses the method until the awaited task completes, without blocking the thread.

Use it for I/O operations: file reading, HTTP calls, database queries.

```
public async Task GetDataAsync()
{
    using var client = new HttpClient();
    string result = await client.GetStringAsync("https://api.example.com");
    return result;
}
```

■ *Exam Tip: Never use .Result or .Wait() on async methods — it causes deadlocks. Always await.*

Exam Duration: 1 hour 31 minutes

itransition | .NET Intern Position

5. ASP.NET Core & REST APIs

NET Development

Q1. What is MVC? Explain each component.

Model: Represents data and business logic. (e.g., a User class, database entity)

View: The UI layer. Displays data from the model. (e.g., .cshtml Razor pages)

Controller: Handles requests, interacts with the model, and returns a view or data.

MVC separates concerns, making code more maintainable and testable.

```
// Controller example
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase {
    [HttpGet]
    public IActionResult GetAll() => Ok(_userService.GetAll());
}
```

C# Fun

Pls

Q2. What are the main HTTP methods and what do they do?

GET: Retrieve data (read-only, safe, idempotent).

POST: Create a new resource.

PUT: Replace an existing resource entirely.

PATCH: Partially update a resource.

DELETE: Remove a resource.

■ *Exam Tip: Idempotent = calling it multiple times gives the same result. GET, PUT, DELETE are idempotent. POST is not.*

Q3. What are common HTTP status codes?

200 OK: Success. **201 Created:** Resource created.

400 Bad Request: Client sent invalid data. **401 Unauthorized:** Not authenticated.

403 Forbidden: Authenticated but not allowed. **404 Not Found:** Resource missing.

500 Internal Server Error: Server-side problem.

■ *Exam Tip: 2xx = success, 3xx = redirect, 4xx = client error, 5xx = server error.*

Q4. What is Dependency Injection (DI) in ASP.NET Core?

DI is a design pattern where a class receives its dependencies from the outside rather than creating them internally.

ASP.NET Core has a built-in DI container. You register services in Program.cs and inject them via constructor.

Service lifetimes: Transient (new each request to the service), Scoped (one per HTTP request), Singleton (one for the app's lifetime).

```
// Register in Program.cs
builder.Services.AddScoped();

// Inject via constructor
public class UsersController {
    private readonly IUserService _svc;
    public UsersController(IUserService svc) { _svc = svc; }
}
```

■ *Exam Tip: DI makes code testable (easy to swap real services with mocks in unit tests).*

6. Entity Framework Core & SQL

.NET Development

Q1. What is an ORM and what is Entity Framework Core?

ORM (Object-Relational Mapper): A tool that lets you interact with a database using C# objects instead of raw SQL.

Entity Framework Core is Microsoft's official ORM for .NET. It maps C# classes to database tables.

With EF Core, you write LINQ queries that get translated into SQL automatically.

```
// C# query instead of SQL
var users = await context.Users
    .Where(u => u.IsActive)
    .OrderBy(u => u.Name)
    .ToListAsync();
```

C# Fun

Pls

itransition | .NET Intern Position

Q2. What is DbContext in Entity Framework Core?

DbContext is the main class for interacting with the database in EF Core.

It represents a session with the database and provides DbSet<T> properties for each table.

It tracks changes to entities and saves them with SaveChangesAsync().

```
public class AppDbContext : DbContext {
    public DbSet Users { get; set; }
    public DbSet Products { get; set; }
    protected override void OnConfiguring(DbContextOptionsBuilder o)
        => o.UseSqlServer(connectionString);
}
```

Q3. What are EF Core Migrations?

Migrations let you evolve your database schema over time as your C# model changes.

Instead of writing SQL ALTER TABLE scripts manually, EF generates them for you.

Commands: Add-Migration [Name] to create a migration, Update-Database to apply it.

```
// Terminal / Package Manager Console:
dotnet ef migrations add AddUserTable
dotnet ef database update
```

■ *Exam Tip: Always review the generated migration file before applying — EF isn't always perfect.*

Q4. What is the difference between SQL JOIN types?

INNER JOIN: Returns rows that match in BOTH tables.

LEFT JOIN: Returns ALL rows from the left table + matched rows from the right (NULL if no match).

RIGHT JOIN: Opposite of LEFT JOIN.

FULL OUTER JOIN: Returns all rows from both tables (NULLs where there's no match).

```
SELECT u.Name, o.OrderDate
FROM Users u
INNER JOIN Orders o ON u.Id = o.UserId
WHERE u.IsActive = 1
```

■ *Exam Tip: INNER JOIN is the most commonly tested. Always know LEFT JOIN too.*

Exam Duration: 1 hour 31 minutes

itransition | .NET Intern Position

7. Quick Reference — Must-Know Answers

.NET Development

Q1. What is the output of this code?

Output: **5**. The method takes 'x' by value, so the original variable 'a' is unchanged.

```
int a = 5;
Void Change(int x) { x = 10; }
Change(a);
Console.WriteLine(a); // Output: 5
```

C# Fun

Q2. What is the difference between String and StringBuilder?

string: Immutable — every modification creates a new object. Bad for many concatenations.

StringBuilder: Mutable — modifies in place. Much faster for building strings in loops.

```
// Slow for many concatenations:
string s = "";
for (int i = 0; i < 1000; i++) s += i; // creates 1000 new strings!

// Fast:
var sb = new StringBuilder();
for (int i = 0; i < 1000; i++) sb.Append(i);
string result = sb.ToString();
```

■ *Exam Tip: Use StringBuilder when concatenating inside a loop.*

Pls

Q3. What is the 'sealed' keyword in C#?

'sealed' on a class prevents it from being inherited.

'sealed' on a method prevents it from being overridden in derived classes.

String class in .NET is sealed — you cannot inherit from string.

```
sealed class FinalClass { } // cannot be inherited
class Base { public virtual void Show() {} }
class Child : Base { public sealed override void Show() {} } // no further overri
de
```


Q4. What is the difference between 'const' and 'readonly'?

const: Compile-time constant. Must be assigned at declaration. Implicitly static.

readonly: Runtime constant. Can be set in declaration OR constructor. Can be instance-level.

```
const double PI = 3.14159;           // compile-time, cannot change
readonly int _maxRetries;             // set once in constructor
public MyClass(int max) { _maxRetries = max; }
```

Complete Q&A with Answers | Intern Level

C# Fundamentals

.NET Core

ASP.NET MVC

LINQ

Entity Framework

REST APIs

Exam Duration: 1 hour 31 minutes

itransition | .NET Intern Position

★. Last-Minute Exam Tips

.NET Development

■ Time Management

You have 1 hour 31 minutes. Quickly scan all questions first. Answer what you know, flag unsure ones, return at the end.

■ Common Code Output Questions

Trace through code mentally step-by-step. Watch for: pass by value vs ref, string immutability, static vs instance scope, and short-circuit evaluation (&&, ||).

■ Focus Areas for Interns

OOP pillars, C# basics, LINQ, DI concept, MVC pattern, HTTP methods & status codes, and basic EF Core usage.

■ If You Don't Know the Answer

The exam note says 'don't worry if you don't know some answers'. Eliminate wrong choices, use logic to narrow down, and make your best guess. Never leave a blank.

■ Keywords That Score Points

Use these in answers: CLR, GC, IDisposable, IEnumerable, LINQ, DbContext, async/await, Dependency Injection, Middleware, REST.